



ASRQuant 1.3.0: Quantitative Research Under Real-World Constraints

Point-in-time data, market conventions, model validation, risk, portfolio construction and backtesting

Alpha Kabinet TOURE Srijan Mishra

Alpha Stochastic Research — Research Team
research@asr-lab.online

ABSTRACT

Quantitative research failures often arise not from a single model equation but from breaks between stages: information may enter a backtest before it was available, calibrated structures may not reproduce their market inputs, strategy selection may be evaluated without accounting for repeated trials, and implementation assumptions may become detached from the reported result. This paper evaluates ASRQuant 1.3.0 as an attempt to make those interfaces explicit within one reproducible quantitative-finance research architecture. The release combines point-in-time data contracts, market conventions and calibration diagnostics, quote-space and credit risk, strategy-selection diagnostics, experiment lineage and chronology-aware backtesting. Evaluation uses deterministic fixtures, fixed-seed simulations, analytical identities, round-trip tests, calibration repricing, numerical invariants and regression tests. A separate internal release-candidate review reproduced and corrected three material numerical issues: key-rate DV01 partitioning on irregular tenor grids, scale consistency in Probabilistic and Deflated Sharpe Ratios, and implied-volatility inversion at the intrinsic boundary. Re-execution of the corrected artifact on Linux x86_64 / Python 3.13.5 produced 336 passing tests with no pytest skips and 77.36% statement coverage across `src/asrquant`; deterministic curve and CDS fixtures reprice to numerical tolerance, and the corrected key-rate construction reconciles aggregate key-rate DV01 with parallel DV01 to approximately $3.91 \times 10^{-6}\%$ relative error on the review fixture. These results support software and research-process claims for the tested artifact, not investment performance or third-party model-validation independence. The principal contribution is therefore architectural: ASRQuant makes data availability, market conventions, calibration quality, model-selection risk, implementation assumptions, provenance and artifact dependencies explicit objects that can be inspected and reproduced across the research chain.

Keywords quantitative finance; fixed income; model validation; point-in-time data; backtesting; reproducibility

Classification JEL: C15, C58, C63, G12

1 Introduction

1.1 Motivation and Problem

Quantitative research is often fragmented across tools that solve individual tasks well but leave the researcher responsible for preserving the links between them. A data series may describe period t yet only become available at $t + 1$; a curve can calibrate numerically while failing to reproduce its input quotes; a strategy can look strong after selection from many trials; a portfolio can appear robust before costs, capacity, or scenario sensitivity are considered; and a backtest can become difficult to reproduce once its data, seed, configuration, and implementation assumptions are separated from the result.

ASRQuant treats this fragmentation as a research-architecture problem. The goal is not to replace specialized market-data systems, pricing libraries, execution platforms, or institutional governance. The goal is to provide a coherent layer in which research inputs, hypotheses, market conventions, quantitative methods, validation evidence, decisions, and artifacts can be inspected together.

Version 1.3.0 extends the 1.2 canonical API without replacing it. The release concentrates on infrastructure: market conventions and schedules, curve diagnostics and quote-space risk, credit foundations, scenario analysis, point-in-time data, strategy-selection diagnostics, experiment lineage, reproducible reports, extensible adapters, and stronger public-API compatibility controls.

1.2 Research Question

The central question is:

Can a unified quantitative-finance platform connect hypothesis discovery to market infrastructure, model validation, portfolio decisions, and backtesting while preserving chronology, provenance, and implementation semantics?

The evaluation therefore asks whether the platform can: (i) distinguish what a datum describes from when it became usable; (ii) calibrate and diagnose market structures rather than merely return an optimizer status; (iii) expose model and strategy-selection risk; (iv) retain reproducible experiment and artifact lineage; and (v) preserve earlier public interfaces while adding these controls.

1.3 Contributions

The 1.3.0 release contributes five connected infrastructure layers.

1. **Market infrastructure.** Calendars, business-day rules, day-count conventions, schedules, rate instruments, curve construction, quote repricing, Jacobians, quote-space risk, rate P&L explanation, credit curves, CDS analytics, and scenarios are exposed through coherent contracts.
2. **Research validation.** CPCV, Probability of Backtest Overfitting, White-style Reality Check, SPA,

leakage diagnostics, consolidated strategy screens, and multiverse/specification analysis complement the pre-existing robustness stack [20, 10, 2, 4, 3, 5, 6].

3. **Point-in-time data and provenance.** Data snapshots, fingerprints, availability/revision timestamps, cache/offline controls, experiment identifiers, and research-artifact dependency graphs make the inputs to a result addressable.
4. **Cross-domain research engineering.** Generic calibration and first-/second-/cross-sensitivities, scenario and transaction-cost models, constrained/cost-aware and robust portfolio tools, covariance-estimator comparison, EVT and filtered-historical tail diagnostics, drawdown-at-risk, copulas, factor/Brinson attribution, volatility and structural-break regimes, model comparison, capacity diagnostics, Monte Carlo variance reduction, and adapter protocols allow specialist components to remain interoperable.
5. **Stability controls.** Native result methods replace the prior compatibility monkey-patching, while a stable public API contract tracks 116 declared symbols, while the release signature inventory records 258 top-level public callables; validation distinguishes local evidence from hosted gates.

2 Related Literature and Positioning

2.1 Quantitative Methods

The platform composes established quantitative methods rather than claiming novelty for their underlying mathematics. Portfolio construction connects to mean-variance analysis [15]; performance evaluation requires care under serial dependence and non-normality [18, 14]; derivative and rate modules build on classical no-arbitrage models [7, 16, 19, 8, 12]; robust covariance and time-series inference draw on standard econometric tools [17, 9, 13].

The 1.3 validation layer is motivated by the risk of selecting strategies after trying many alternatives. Reality Check, SPA, Deflated Sharpe, PBO, and multiple-testing evidence motivate an explicit distinction between exploration and confirmation [20, 10, 1, 2, 11]. The software therefore treats statistical evidence as one component of a research decision rather than as an automatic strategy approval.

2.2 Architecture Positioning

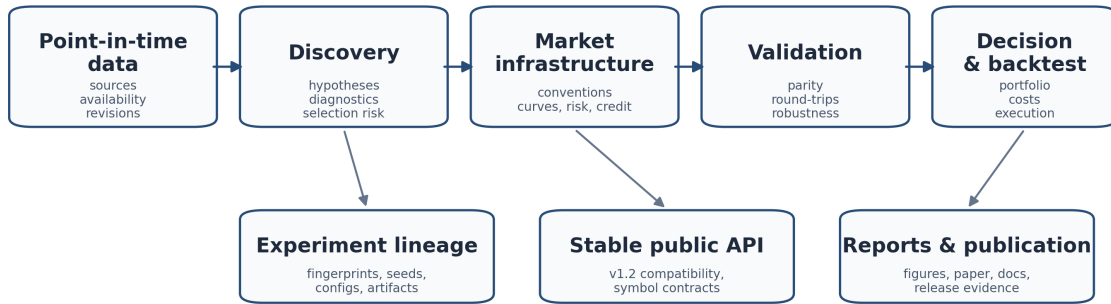
The differentiating object is the *research chain*. ASRQuant is designed so that a team can use its own data provider, pricing engine, or cost model while retaining common experiment, validation, provenance, and reporting contracts. This reduces the incentive to force every institution onto a single infrastructure stack and makes interoperability a first-class requirement. The comparison below is descriptive rather than a ranking; the projects have different primary scopes.

Table 1: Mapping from the five research-question criteria to operational tests and manuscript evidence.

Criterion	Operational test	Primary evidence
(i)	Point-in-time chronology	Availability/revision fixtures; point-in-time timeline; chronology and leakage tests.
(ii)	Calibration diagnostics	Exact curve-quote and CDS repricing; Jacobian and quote-risk checks; Figs. 5–7.
(iii)	Model and selection risk	CPCV, PBO, Reality Check, SPA, PSR/DSR and multiverse tests; validation diagnostics.
(iv)	Reproducible lineage	Snapshot fingerprints, experiment identifiers and artifact-dependency tests; architecture and provenance figures.
(v)	Interface preservation	Stable-API, callable-signature and release-integrity checks; Sec. 9.

Research-chain view of ASRQuant 1.3.0

Chronology, market semantics, native validation and reproducible artifacts are treated as one connected workflow.

**Figure 1:** ASRQuant 1.3.0 organizes the research chain around point-in-time data, discovery, market infrastructure, validation, decision, and lineage/reporting layers.

Project	Primary emphasis and relation to ASRQuant
QuantLib	Broad quantitative-finance modelling, pricing and risk library. ASRQuant overlaps in rates and derivatives while placing point-in-time research lineage and release-validation contracts at the centre of the workflow.
OpenBB	Financial-data and research workspace/integration layer. ASRQuant overlaps in data workflows while embedding finance/statistical engines and audit-oriented research objects in the same package.
vectorbt	High-performance vectorized strategy analysis and backtesting. ASRQuant overlaps in backtesting and validation while also covering market calibration, fixed-income/credit infrastructure and artifact lineage.

3 Data, Point-in-Time Information and Provenance

3.1 Data Sources and Quality

The canonical data layer continues to accept pandas objects, local tabular files, public HTTP(S) tables, and provider-backed data including Yahoo Finance, ECB, FRED, Binance, and Alpha Vantage. Validation reports missingness, duplicate indices, non-finite values, constant columns, ordering, and sampling gaps without silently rewriting the input.

Version 1.3 adds deterministic snapshots and a point-in-time contract. Let t_o denote the observation timestamp, t_a the first timestamp at which the value was available to the researcher, and t_r a later revision timestamp. A valid research view at decision time t may use an observation only if $t_a \leq t$. This prevents the common error of treating revised historical data as if the revised value had been known in real time.

3.2 Snapshots and Experiment Identity

A data snapshot stores a deterministic fingerprint, source metadata, schema, creation time, and serialized representation. A local `DataStore` adds keyed caching, freshness checks, and offline replay of previously cached data. An experiment combines named configuration, data fingerprints, a seed, optional code hash, package/Python version, and environment metadata. The resulting fingerprint can be registered in an append-only local registry and used as a stable reference for downstream artifacts. `ResearchReport` can combine heterogeneous result objects into HTML, Markdown, or JSON. If an upstream artifact changes, a research graph identifies dependent artifacts that should be considered stale.

```

snapshot = store.put("rates_panel", data, source="
internal")
experiment = asr.research.Experiment(
    name="curve_signal",
    config={"lookback": 60, "cost_bps": 5},
    data={"market": data}, seed=42,
)

```

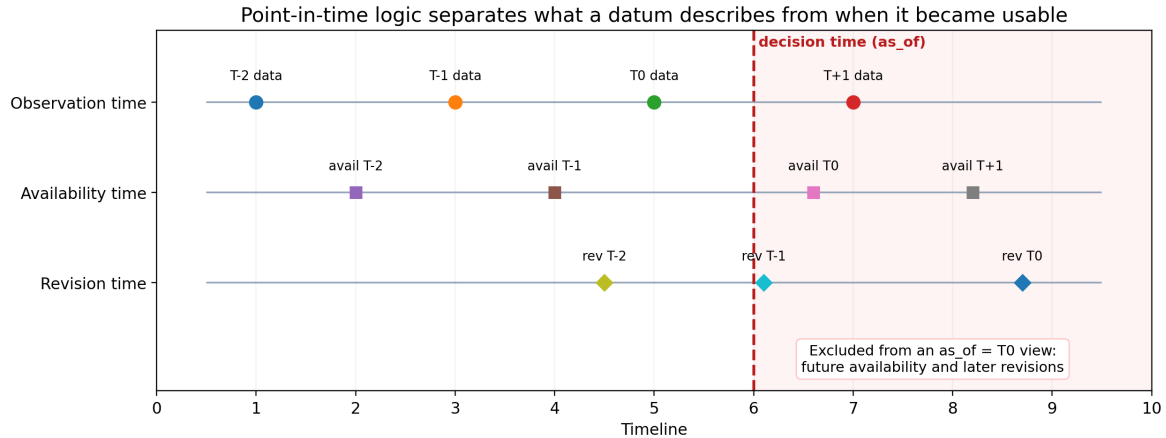


Figure 2: Point-in-time data separates observation time, availability time, and revision time. An `as_of` view includes only information available by the decision timestamp.

4 Hypothesis Discovery and Identification

4.1 Evidence and Novelty

The hypothesis system introduced in earlier releases remains intact. Candidates can arise from data, literature, model disagreement, robustness instability, structural changes, regimes, or domain-specific observations. The core rule remains conservative: a statistically supported candidate is not automatically novel, and a corpus-relative research gap is not proof of global novelty.

Evidence status and novelty status are therefore represented separately. This separation lets a candidate be out-of-sample supported yet already documented in prior work, or potentially novel while still statistically inconclusive.

4.2 Failure Conditions

A candidate should be weakened or rejected when its sign changes materially across reasonable specifications, its holdout evidence disappears, a chronology correction removes the effect, multiple-testing adjustment eliminates the finding, an alternative benchmark explains the result, or prior literature establishes that the purported mechanism is already known. The platform records these outcomes rather than forcing all candidates toward a positive decision.

5 Market Infrastructure

5.1 Conventions, Schedules and Instruments

Version 1.3 adds a common conventions layer for TARGET, US federal, UK bank, and weekend-only calendars; custom holidays; business-day adjustments; day counts; stubs; frequencies; fixing/payment lags; accrual fractions; and schedules. These are not cosmetic details: they determine accrual fractions, cash-flow dates, discounting intervals, and hedge sensitivities. The high-level instrument layer covers deposits, FRAs, fixed-rate bonds, swaps, and OIS objects with explicit pricing, cash-flow, and risk methods, so that rate workflows do not rely on ad hoc date arithmetic in each notebook.

5.2 Curve Construction and Diagnostics

A curve build is treated as a constrained numerical object with explicit diagnostics. Given market quotes q and internal curve parameters z , the builder retains the market quotes and returns a curve, quote-repricing errors, calibration diagnostics, and a quote-to-zero Jacobian $\partial z / \partial q$. Sparse swap maturities can use an explicit `grid_policy=interpolate`, with synthetic coupon-grid nodes recorded in metadata, while `grid_policy=strict` requires a completely supplied coupon grid. The test fixture used in this paper combines short deposits and par swaps. The calibrated curve reproduces the input quotes to numerical tolerance while preserving positive discount factors.

5.3 Quote-Space Risk and P&L Explain

Risk is often consumed in market-quote space rather than in an arbitrary internal zero-rate grid. By the chain rule,

$$\frac{\partial V}{\partial q} = \frac{\partial V}{\partial z} \frac{\partial z}{\partial q}. \quad (1)$$

ASRQuant exposes node risk and quote risk separately. For a par swap, the deterministic example concentrates most quote PV01 in the terminal swap quote, as expected from the calibration structure of the sparse fixture.

Rate P&L explanation then compares instrument values under two curve states and attributes the move using the same explicit curve objects. This provides a consistent path from market inputs to valuation, sensitivities, scenarios, and subsequent review.

6 Credit, Scenarios and Cross-Domain Risk

6.1 Credit Foundations

Version 1.3 introduces piecewise-constant hazard and survival curves together with default probabilities, CDS premium/protection-leg analytics, par spreads, CS01-style hazard sensitivity, jump-to-default diagnostics, and sequential hazard bootstrapping. For piecewise hazard

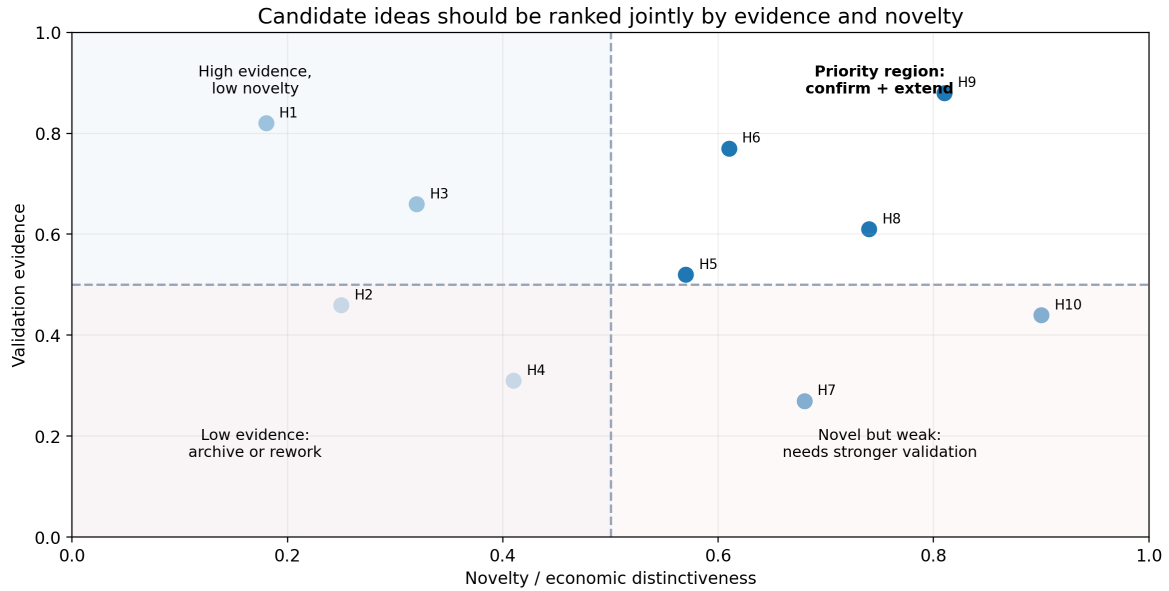


Figure 3: Illustrative candidate states showing statistical evidence and novelty as separate dimensions.

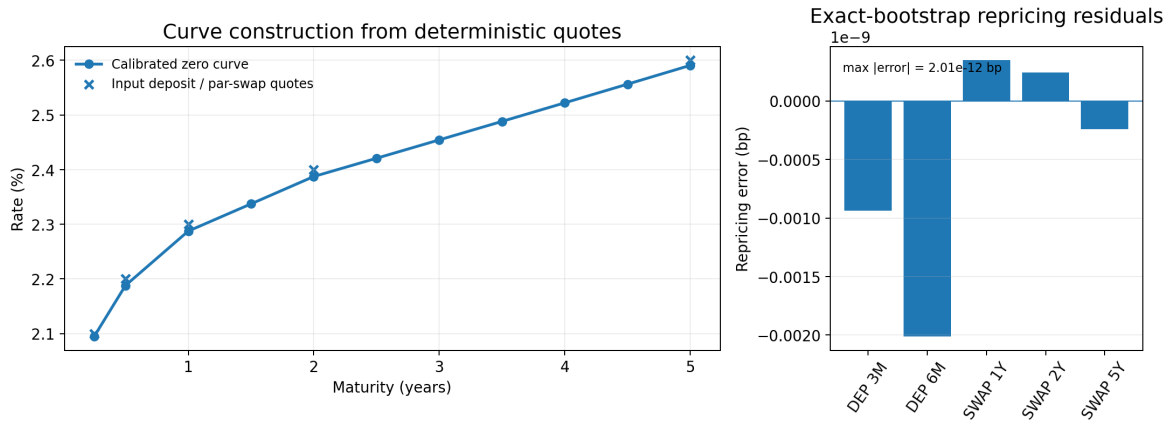


Figure 4: Deterministic 1.3 curve-builder example. The displayed maximum quote-repricing error is numerical rather than economic.

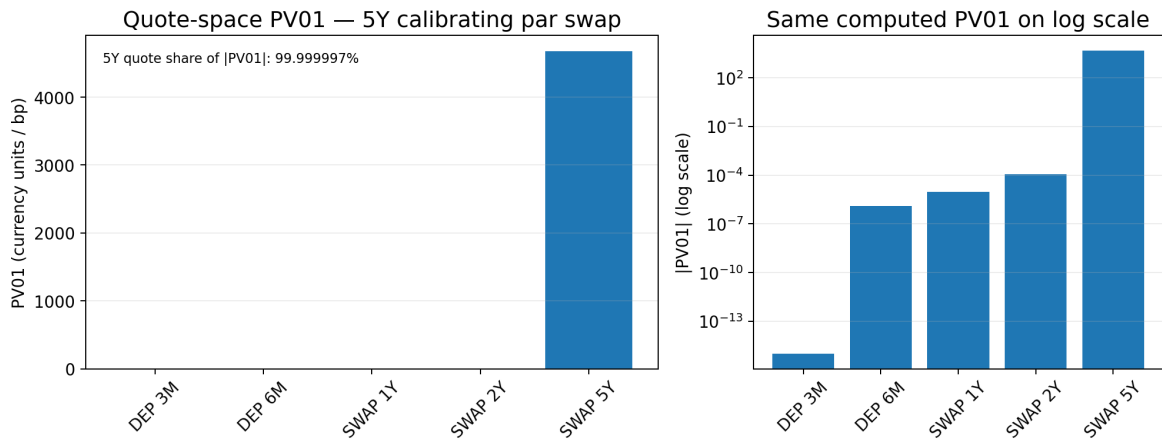


Figure 5: Quote-space PV01 for an illustrative 5Y par swap calibrated to the deterministic market-quote set.

rate $\lambda(t)$, survival is

$$Q(0, T) = \exp \left(- \int_0^T \lambda(u) du \right). \quad (2)$$

The deterministic example bootstraps a hazard curve from 1Y, 3Y, and 5Y CDS spreads under a fixed recovery

assumption. The purpose is infrastructure validation, not an empirical credit forecast.

6.2 Scenario Engine

A common scenario object can represent parallel, slope, curvature, volatility, liquidity, or asset shocks and route them to compatible instruments or portfolios. This

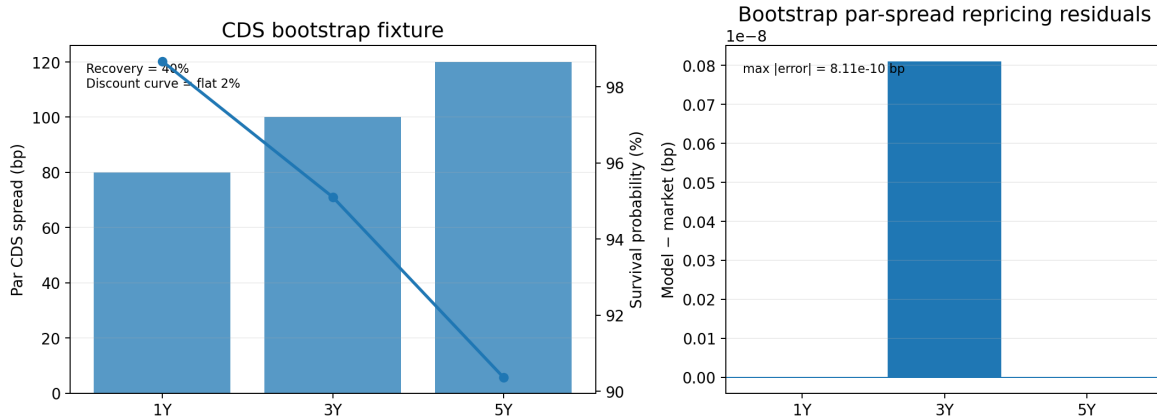


Figure 6: Illustrative survival probabilities implied by the deterministic CDS bootstrap fixture.

allows scenario assumptions to be named, stored, compared, and attached to research reports instead of remaining embedded in isolated notebook cells.

6.3 Cross-Domain Research Utilities

The 1.3 utility layer also includes generic calibration with residual, Jacobian, and conditioning diagnostics; first-, second-, and cross-sensitivity estimation; fixed, spread, linear-impact, and square-root-impact cost models; EVT tail modelling, filtered historical simulation, and drawdown-at-risk; Gaussian and Student- t copulas; factor and Brinson attribution; volatility and structural-break regime diagnostics; model and covariance-estimator comparison; Monte Carlo variance-reduction helpers; rate P&L explanation; signal-portfolio capacity diagnostics; and adapter protocols for data, pricing, risk, execution, cost, and optimizer components.

7 Research Validation

7.1 Selection Risk and Chronology

A strategy that is selected from many alternatives has a different evidential status from a strategy specified before looking at the data. ASRQuant 1.3 therefore adds combinatorial purged cross-validation, PBO, Reality Check, SPA, leakage diagnostics, and consolidated strategy reports. These diagnostics complement rather than replace economic reasoning.

7.2 Multiverse and Leakage Diagnostics

Multiverse analysis evaluates a research conclusion across multiple reasonable choices such as lookback, normalization, neutralization, cost assumptions, or model specification. Leakage diagnostics search for observable chronology violations including future shifts, unsafe train/test overlap, and transformations that use information outside the training window. Neither tool proves validity; both make fragility easier to identify before a result is promoted.

8 Portfolio, Costs and Backtesting

8.1 Decision Layer

The existing portfolio layer remains available for minimum variance, maximum Sharpe, risk budgeting, maximum diversification, and hierarchical risk parity, with 1.3 additions for constrained/cost-aware optimization, robust mean-variance construction, risk budgeting, and covariance-estimator comparison. Fixed, spread, linear-impact, and square-root-impact transaction-cost models and scenario objects can be passed into the same decision workflow, reducing the gap between an unconstrained research optimum and an implementable allocation.

8.2 Backtest as Implementation Contract

The backtesting layer continues to require explicit chronology. Let w_t be effective asset weights, r_{t+1} next-period returns, and C_{t+1} implementation cost. A minimal net return identity is

$$r_{t+1}^{net} = w_t^\top r_{t+1} - C_{t+1}. \quad (3)$$

The important property is not the formula itself but that delay, turnover, costs, cash, borrow, and missing-data assumptions are part of the recorded implementation rather than post-hoc adjustments.

9 Experimental Design and Release Validation

9.1 Validation Design

The paper evaluates software behavior, not a market anomaly. Tests therefore use deterministic fixtures, fixed random seeds, mocked provider responses, known pricing identities, and controlled synthetic data. Numerical controls include quote repricing, discount-factor/zero-rate round trips, par-swap zero-PV identity, CDS bootstrap repricing, Black-Scholes put-call parity, finite-difference sensitivities, positive-semidefinite covariance checks, and deterministic experiment fingerprints.

The review re-execution collected 336 tests across 36 modules and completed with 336 passes and no pytest skips. Tests were run in isolated groups because plotting, solver, and numerical backends can otherwise share process state. Combining the grouped coverage data gives 77.36% statement coverage across `src/asrquant` (13,092 statements; 2,964 missed). Coverage is reported

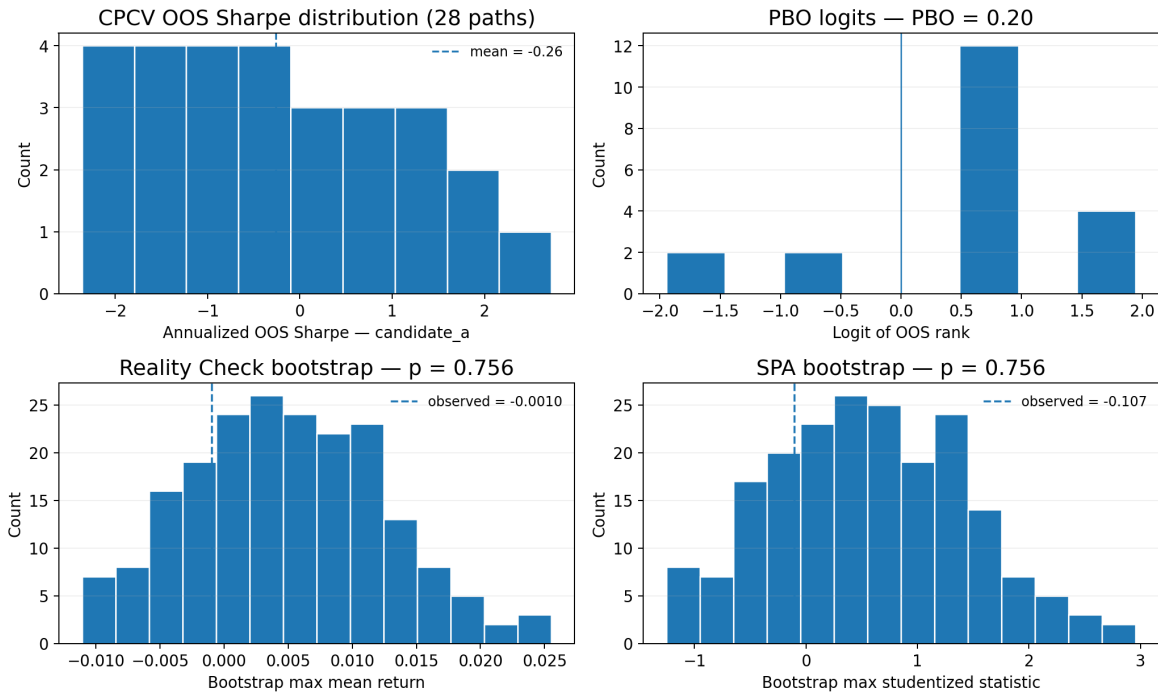


Figure 7: Deterministic synthetic example showing PBO, Reality Check, and SPA diagnostics. The values demonstrate API behavior only.

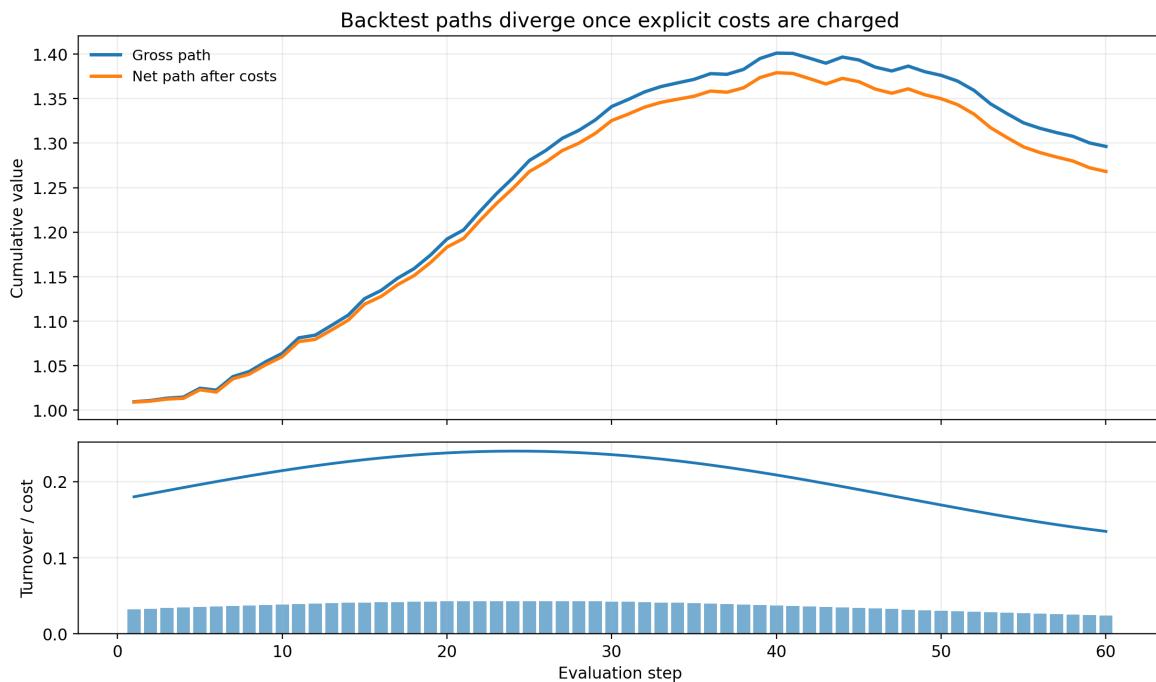


Figure 8: Deterministic synthetic illustration showing how explicit costs change the backtest path.

as scope evidence rather than as a correctness score.

9.2 Distribution and API Controls

The package import sweep reports 84 package modules imported with zero failures, excluding `asrquant.__main__` because importing it intentionally executes the CLI. A stable API snapshot tracks 116 declared symbols, while a separate signature inventory records 258 top-level public callables; automated compatibility checks fail when a stable symbol disappears or its callable signature changes unexpectedly. Canonical result serialization is native in the final artifact. The

supplied wheel and source distribution were rebuilt from the corrected source tree, and a wheel smoke installation reports version 1.3.0. The CLI and dataset-validation surfaces remain covered by the package test suite and release checks.

The public repository CI defines a 12-cell compatibility matrix spanning Linux, macOS and Windows with Python 3.10–3.13, plus domain, quality, security and distribution jobs. This manuscript reports only the local Linux/Python 3.13.5 re-execution for the supplied artifact. The exact 1.3.0 public release commit must complete the hosted matrix before publication; an earlier

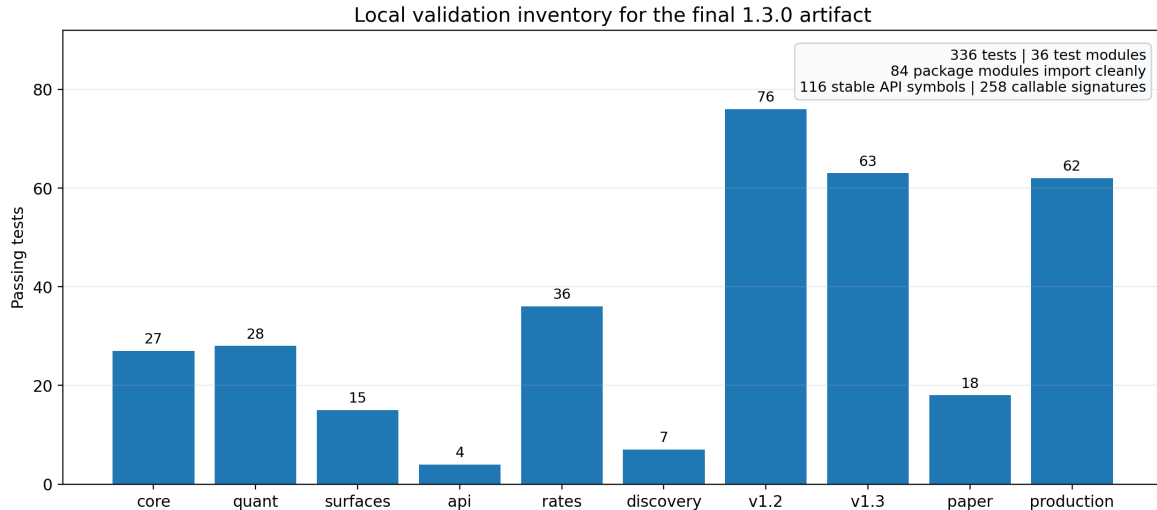


Figure 9: Local ASRQuant 1.3.0 validation inventory. Counts are taken from the release validation record.

or different commit is not treated as evidence for this artifact.

10 Release-Candidate Review Corrections and Numerical Integrity

10.1 Why the release-candidate review matters

A numerical library can pass a large internal test suite and still carry a localised convention error. The final 1.3.0 release therefore treats the release-candidate review as a separate validation workstream. That workstream was led by co-author Srijan Mishra and covered numerical correctness, statistical conventions, public interfaces, reproducibility, edge cases, defect reproduction, regression protection and release-readiness evidence. It was organizationally internal to ASR, not third-party independent model validation. Separation here means that identified defects were reproduced independently from the implementation change and challenged through explicit tests; it does not imply institutional independence. A finding is incorporated only when its reproduction identifies a concrete contract, convention, numerical boundary, or dependency behaviour. Statistical estimation limits are not relabelled as software defects merely because their estimates can be noisy.

10.2 Key-rate DV01 on uneven market pillars

The most material rates finding concerned triangular key-rate bumps. A symmetric triangle built from the larger neighbouring spacing is harmless on uniform grids but overlaps on an irregular market pillar set. If $w_i(t)$ denotes the bump weight associated with pillar i , a first-order partition requires

$$\sum_i w_i(t) = 1 \quad (4)$$

through the supported curve grid. The final implementation uses the actual left and right gaps separately. On the standard uneven pillar set $[0.25, 0.5, 1, 2, 3, 5, 7, 10, 15, 20]$, this removes the spurious risk spillover that could otherwise make key-rate DV01s fail to reconcile with a parallel DV01. Automatic

key-rate bumps now also require the selected maturity to coincide with a curve pillar; arbitrary centres remain available only when the caller supplies an explicit width.

10.3 Probabilistic and Deflated Sharpe Ratios

The Probabilistic Sharpe Ratio is defined on a per-observation Sharpe statistic. Feeding an annualised Sharpe into the same finite-sample formula mixes scales with the benchmark and makes the multiple-testing deflation too weak. The final implementation retains the public annualisation parameter for compatibility but computes the PSR statistic from the per-observation Sharpe. DSR then compares like with like. This correction is especially important because the result is a probability-like quantity: a scale error can look plausible rather than fail loudly.

10.4 Non-identifiable implied volatility

When a Black-style option quote lies at the discounted intrinsic boundary, floating-point arithmetic can make the objective exactly zero at the solver's lower volatility bracket. Returning that lower bracket creates a false impression of a successful calibration. The final implementation explicitly evaluates the bracket boundary and refuses the inversion when the quote is at the non-identifiable boundary. Interior quotes still use bracketed root finding and preserve the existing round-trip behavior.

10.5 Stable PCA orientation, statistics compatibility and explicit conventions

Principal-component eigenvectors are defined only up to sign. The final yield-curve PCA orients each component using its largest absolute loading, preventing arbitrary sign inversions from contaminating rolling factor interpretation while leaving reconstruction unchanged. Removed statsmodels 0.15 keywords are no longer passed to AutoReg or Granger-causality functions. RSI-14 now defaults to Wilder smoothing, while the alternate simple-moving-average convention remains explicitly selectable. Gaussian-process observation noise is fixed when supplied by the user and the requested and fitted values are

both reported. Vasicek calibration now reports standard errors for the AR(1) coefficient and the implied mean-reversion speed, making the well-known short-sample uncertainty visible rather than pretending that calendar-span limitations can be solved by denser sampling.

10.6 Dependency architecture

The final core installation is smaller than the RC dependency surface. Matplotlib/Plotly/Jinja2, scikit-learn, requests, and pypdf are behind explicit extras and are imported only when their capabilities are invoked. A fresh `import asrquant` therefore does not itself load Matplotlib, requests, or scikit-learn. The purpose is not merely import speed: regulated, offline, or minimal environments can audit a smaller dependency graph and add specialised stacks intentionally.

10.7 Before/after numerical evidence

Table 2 reconstructs the two scale-sensitive release-candidate behaviours using deterministic review fixtures. For key-rate risk, the RC rule uses the larger adjacent spacing for both sides of each triangular bump; the final rule uses the actual left and right spacings. For PSR/DSR, the RC calculation inserts an annualized Sharpe into a per-observation finite-sample statistic; the final implementation keeps the statistic and benchmark on the same scale. Signed DV01 values depend on payer/receiver convention, so reconciliation is assessed by the difference between parallel DV01 and the sum of key-rate DV01s.

Table 2: Deterministic before/after numerical evidence used in the release-candidate review.

Metric	RC rule	Final 1.3.0
Max. $ \sum_i w_i - 1 $ on uneven grid	0.7500	0.0000
Parallel DV01 (currency/bp)	-15,152.700	-15,152.700
Sum key-rate DV01 (currency/bp)	-15,441.509	-15,152.699
$ \sum KR - DV01 $ (currency/bp)	288.809	5.92×10^{-4}
Relative KR reconciliation error	1.906%	$3.91 \times 10^{-6}\%$
PSR, benchmark 0	1.0000	0.8411
DSR, 100 trials	1.0000	0.0636

The PSR/DSR row uses a fixed 252-observation synthetic series with per-observation Sharpe 0.0632 and annualized Sharpe 1.0034; it is intentionally chosen to expose the scale error rather than produce a saturated post-fix probability.

11 Native Quantitative Engines and Numerical Independence

ASRQuant keeps its finance-specific model implementations inside the package rather than delegating pricing, curve construction, rate risk, calibration or derivative analytics to an external quantitative-finance engine. This design makes model equations, conventions and numerical boundaries inspectable in the same source tree as the research workflow that uses them.

Independence does not mean avoiding the scientific Python ecosystem. NumPy, SciPy, pandas and statsmodels provide general numerical, array, data and econometric primitives where appropriate. The distinction is

that these libraries do not substitute a separate finance-pricing engine for ASRQuant’s own instrument and risk logic.

Validation therefore emphasizes model-specific mathematical contracts. European option routines are checked by put–call parity, implied-volatility round trips and an independently coded Black–Scholes formula; curve and rate-risk routines are checked by repricing and partition identities where applicable; stochastic and statistical routines are checked against known moments, invariants, deterministic seeds, limiting cases and regression fixtures.

Control	Observed error	Evidence
Black–Scholes closed form	1.421×10^{-14} max abs.	Three independently coded formula comparisons.
CRR, 1000 steps	0.00199947 max abs.	Convergence to Black–Scholes; tolerance 0.01.
Curve quote bootstrap	2.01×10^{-12} bp max	Deterministic deposit/swap fixture; curve figure.
CDS bootstrap	8.11×10^{-10} bp max	1Y/3Y/5Y CDS fixture; credit figure.
External QuantLib / ISDA	not claimed	No SW-007 reference output is present in the supplied artifact or public default branch.

The absence of SW-007 evidence is stated explicitly rather than replaced with inferred QuantLib or ISDA numbers. Such external reference comparisons can be appended when the exact result file and environment are supplied. The release-candidate review complements those contracts by reproducing edge cases and challenging conventions at the API boundary.

The design target is a self-contained quantitative research platform whose finance-specific behavior can be inspected, tested and extended directly in ASRQuant. Claims of correctness are tied to explicit numerical contracts and observed test evidence rather than to agreement with an external finance library.

12 Results

12.1 Primary Result

The main result is architectural rather than predictive. ASRQuant 1.3.0 can represent a continuous research path from point-in-time inputs and hypothesis candidates through market calibration, validation, scenarios, portfolio decisions, and auditable backtesting while recording the provenance and dependencies of intermediate artifacts.

CLAIM 01

LOCAL RELEASE EVIDENCE

On Linux x86_64 / Python 3.13.5, all defined local release gates for the corrected artifact passed, including the 336-test suite, import/API/release checks and wheel smoke import. The grouped test re-execution recorded no pytest skips and 77.36% source statement coverage. These are software-validation results, not evidence of investment performance.

Local validation supports the corrected final artifact in the tested environment: 336 tests pass across 36 test

modules; 84 package modules import without failure; the stable API contract covers 116 declared symbols and the signature inventory covers 258 top-level callables; the public-API and release-version checks pass; and the rebuilt wheel smoke installation reports version 1.3.0.

12.2 Interpretation

The release should therefore be read as a strengthening of quantitative research infrastructure. It does not establish that every model or strategy produced with the package is correct. Instead, it makes more of the assumptions that determine correctness visible: when data were available, what market conventions were used, whether calibration reproduced market quotes, how quote risk propagates, how many strategy alternatives were considered, whether chronology was violated, and which artifacts depend on which inputs.

13 Robustness, Limitations and Implementation Boundaries

13.1 Robustness Controls

The 1.3 tests exercise both normal and failure paths, including sparse/complete curve grids, randomized option parity, credit repricing, multiverse execution, point-in-time revisions, portfolio constraints, cost positivity, Monte Carlo variance reduction, regime diagnostics, and artifact-lineage propagation. API compatibility and distribution-install checks protect a different failure mode: a mathematically correct implementation that is difficult to install or unexpectedly breaks existing notebooks.

13.2 Limitations

The release is not a replacement for venue-specific holiday databases, institutional entitlements, a complete credit/XVA stack, exchange execution infrastructure, or independent model-validation governance. The point-in-time contract can only be as reliable as the availability metadata supplied to it. Multiple-testing diagnostics depend on the candidate set presented to the test. Curve and credit examples in this manuscript are simplified deterministic fixtures. Scenario and cost models are modelling assumptions, not observed future execution conditions. Guarded live-broker components remain separate from software-release status and are designed to fail closed.

The current local evidence is platform-specific: Linux x86_64 and Python 3.13.5. Cross-platform package publication should therefore preserve the hosted compatibility matrix as a separate release gate; the manuscript does not report that gate as locally observed.

14 Economic and Research Implications

For a researcher, the practical benefit is fewer silent translations between stages. For a quantitative team, the adapter layer means internal providers, pricers, or desk cost models can be registered without abandoning the common research workflow. For review, fingerprints and lineage make it easier to answer which data, con-

figuration, code version, and seed generated an artifact. For model governance, diagnostics create explicit places to record inconclusive, unstable, or failed results rather than forcing every workflow into a successful numeric output.

The intended standard is therefore not feature count. A capability belongs in the platform when it improves correctness, traceability, validation, interoperability, or decision relevance without weakening the stable public API.

15 Conclusion

ASRQuant 1.3.0 extends the platform from a broad quantitative toolkit toward a more integrated research infrastructure. The release strengthens the bridge between hypothesis discovery and backtesting with point-in-time data, market conventions, curve diagnostics, quote-space risk, credit foundations, scenarios, advanced strategy validation, experiment fingerprints, lineage, and cross-domain reports.

The local release evidence supports a precise conclusion: the corrected 1.3.0 artifact passes the defined local validation in the tested environment, with 336 passing tests, 84 package-module imports, 116 stable API symbols, 258 recorded top-level callable signatures, a passing public-API check, a passing release-version check, and a successful wheel smoke import. These results do not by themselves establish behavior on untested operating systems, Python versions, brokers or external data services.

The broader contribution is architectural. Quantitative research becomes easier to challenge when data availability, model conventions, selection risk, implementation costs, and artifact dependencies are represented as explicit objects rather than implicit notebook knowledge. That is the design direction of ASRQuant 1.3.0.

Data and code availability

The complete ASRQuant 1.3.0 source artifact accompanying this manuscript contains the source package, tests, examples, documentation, release metadata, wheel and source distribution. The project is MIT licensed. Figures use deterministic repository fixtures or fixed-seed synthetic data; no live market feed is required to reproduce the software-validation evidence reported here. Public registry publication is a separate release action.

Reproducibility statement

The reported local evidence is tied to the corrected 1.3.0 artifact and a Linux x86_64 / Python 3.13.5 environment. The release record reports 336 passing tests across 36 modules, 84 package modules imported with zero failures, a passing public-API compatibility check, a passing release-version check and a successful wheel smoke import. Hosted multi-OS, registry and external-service checks are not presented as observed results in this manuscript.

Author contributions

Alpha Kabinet TOURE: conceptualization, overall software architecture, core implementation, methodology, release integration, visualization, project administration, writing—original draft, and manuscript preparation. **Srijan Mishra:** lead release-candidate validation and benchmarking; design and execution of the validation methodology; numerical, statistical, interface and public-API review; reproducibility methodology and review; defect identification, reproduction, triage and verification; regression-test design and review; benchmark construction and interpretation; robustness, boundary-condition and edge-case assessment; review of corrective implementations; validation evidence and release-readiness assessment; interpretation of validation results; substantive scientific review of the manuscript; co-writing of the Validation Methodology, Experimental Setup/Benchmarking, Results, Discussion and Limitations material; and writing—review and editing.

Funding

No external funding was declared for this manuscript.

Competing interests

Alpha Kabinet TOURE is a co-founder of Alpha Stochastic Research, the organization that develops and publishes ASRQuant. This relationship is disclosed because the manuscript evaluates an ASR software artifact. Srijan Mishra led the internal release-candidate validation and benchmarking workstream and contributed to scientific review and manuscript development. No additional competing interests are declared.

Artificial-intelligence use disclosure

Artificial-intelligence tools were used solely to assist with the final drafting and editorial polishing of this document.

Acknowledgements

The authors acknowledge ASR contributors and the scientific Python ecosystem used by the project. Release-candidate validation submissions materially informed the numerical, compatibility and packaging corrections documented in this paper.

Publication metadata

Version: 1.3.0

Repository:

<https://github.com/Alpha-Stochastic-Research/asr-quant>

DOI: Pending Zenodo deposit

References

- [1] D. H. Bailey and M. López de Prado. The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality. *Journal of Portfolio Management*, 40(5):94–107, 2014. doi:10.3905/jpm.2014.40.5.094.
- [2] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu. The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4):39–69, 2017. doi:10.21314/JCF.2016.322.
- [3] D. H. Bailey and M. López de Prado. The Sharpe Ratio Efficient Frontier. *Journal of Risk*, 15(2):3–44, 2012.
- [4] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- [5] S. Steegen, F. Tuerlinckx, A. Gelman, and W. Vanpaemel. Increasing Transparency Through a Multiverse Analysis. *Perspectives on Psychological Science*, 11(5):702–712, 2016. doi:10.1177/1745691616658637.
- [6] U. Simonsohn, J. P. Simmons, and L. D. Nelson. Specification Curve Analysis. *Nature Human Behaviour*, 4:1208–1214, 2020. doi:10.1038/s41562-020-0912-z.
- [7] F. Black and M. Scholes. The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*, 81(3):637–654, 1973. doi:10.1086/260062.
- [8] J. C. Cox, J. E. Ingersoll Jr., and S. A. Ross. A Theory of the Term Structure of Interest Rates. *Econometrica*, 53(2):385–407, 1985. doi:10.2307/1911242.
- [9] R. F. Engle and C. W. J. Granger. Co-integration and Error Correction: Representation, Estimation, and Testing. *Econometrica*, 55(2):251–276, 1987. doi:10.2307/1913236.
- [10] P. R. Hansen. A Test for Superior Predictive Ability. *Journal of Business & Economic Statistics*, 23(4):365–380, 2005. doi:10.1198/073500105000000063.
- [11] C. R. Harvey, Y. Liu, and H. Zhu. ... and the Cross-Section of Expected Returns. *Review of Financial Studies*, 29(1):5–68, 2016. doi:10.1093/rfs/hhv059.
- [12] S. L. Heston. A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. *Review of Financial Studies*, 6(2):327–343, 1993. doi:10.1093/rfs/6.2.327.
- [13] O. Ledoit and M. Wolf. A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004. doi:10.1016/S0047-259X(03)00096-4.
- [14] A. W. Lo. The Statistics of Sharpe Ratios. *Financial Analysts Journal*, 58(4):36–52, 2002. doi:10.2469/faj.v58.n4.2453.
- [15] H. Markowitz. Portfolio Selection. *Journal of Finance*, 7(1):77–91, 1952. doi:10.1111/j.1540-6261.1952.tb01525.x.
- [16] R. C. Merton. Theory of Rational Option Pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973. doi:10.2307/3003143.
- [17] W. K. Newey and K. D. West. A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*, 55(3):703–708, 1987. doi:10.2307/1913610.
- [18] W. F. Sharpe. Mutual Fund Performance. *Journal of Business*, 39(S1):119–138, 1966. doi:10.1086/294846.

- [19] O. Vasicek. An Equilibrium Characterization of the Term Structure. *Journal of Financial Economics*, 5(2):177–188, 1977. doi:10.1016/0304-405X(77)90016-2.
- [20] H. White. A Reality Check for Data Snooping. *Econometrica*, 68(5):1097–1126, 2000. doi:10.1111/1468-0262.00152.

Disclosure and disclaimer

Alpha Stochastic Research publications are produced for scientific and educational purposes. This paper documents software and research methodology; it does not constitute investment advice, an offer, solicitation or recommendation to transact in any financial instrument.

A Mathematical and Implementation Identities

A.1 Quote-space sensitivity

If an instrument value V depends on internal curve nodes z and those nodes depend on market quotes q , first-order quote sensitivity is

$$\nabla_q V = (\nabla_z V) \frac{\partial z}{\partial q}. \quad (5)$$

The distinction matters because a portfolio is usually hedged and reported against traded quotes rather than an arbitrary interpolation grid.

A.2 Credit survival

For piecewise-constant hazard rate λ_k on intervals $(t_{k-1}, t_k]$,

$$Q(0, T) = \exp \left(- \sum_k \lambda_k \Delta t_k \right), \quad (6)$$

with intervals truncated at T . The bootstrap chooses hazard segments so the CDS premium and protection legs reproduce the supplied market spreads under the stated recovery convention.

A.3 Backtest net-return identity

With effective risky weights w_t , next-period asset returns r_{t+1} , residual cash return r_{t+1}^f , and implementation cost C_{t+1} ,

$$r_{p,t+1}^{net} = w_t^\top r_{t+1} + (1 - \mathbf{1}^\top w_t) r_{t+1}^f - C_{t+1}. \quad (7)$$

This identity makes residual cash and explicit implementation cost part of the recorded backtest contract.

B Canonical API Map

Namespace	Representative 1.3 role
<code>asr.data</code>	loading, validation, snapshots, point-in-time views
<code>asr.hypotheses</code>	discovery, search, novelty audit, project handoff
<code>asr.rates</code>	conventions, schedules, curves, instruments, quote risk, P&L explain
<code>asr.credit</code>	hazard/survival curves, CDS analytics and bootstrap
<code>asr.validation</code>	CPCV, PBO, Reality Check, SPA, leakage, multiverse
<code>asr.scenarios</code>	named asset/rate/volatility/liquidity shocks
<code>asr.portfolio</code>	optimization, robust/cost-aware construction, risk budgeting
<code>asr.risk</code>	VaR/ES, tail diagnostics, covariance comparison
<code>asr.alpha / factors</code>	signal diagnostics, capacity, exposures and attribution
<code>asr.backtesting</code>	chronology-aware implementation and cost accounting
<code>asr.research</code>	experiments, registry, lineage graph, reports
<code>asr.register</code>	adapters for internal providers, pricers and cost models

C Extended Release Evidence

The local validation record reports 336 tests across 36 test modules: core 27, quant 28, surfaces 15, API 4, rates 36, discovery 7, v1.2 regression 76, v1.3 63, paper 18, and production 62. The v1.3 audit-regression file contains direct tests for the corrected numerical/interface contracts. The package import sweep covers 84 modules with zero failures. The stable public compatibility snapshot covers 116 declared symbols and the callable-signature inventory records 258 top-level public callables. The corrected wheel was rebuilt from this source tree and a smoke installation reports version 1.3.0.

The evidence reported here is local-artifact evidence. Hosted Python 3.10–3.13 and Linux/macOS/Windows compatibility jobs, hosted lint/type/security gates, a clean PEP 517 build, strict Twine validation, and TestPyPI installation are separate operational gates when the distribution is published to a registry.

D Release-Update Coverage Matrix

This appendix provides an explicit audit against the ASRQuant 1.3.0 release notes and changelog. Every declared 1.3.0 update is represented in the manuscript:

- **Market conventions:** calendars, custom holidays, business-day rules, day counts, stubs, frequencies, schedules, fixing/payment lags, and accrual fractions (Market Infrastructure section).
- **Rate instruments:** deposits, FRAs, fixed-rate bonds, swaps, and OIS with explicit pricing/cash-flow/risk methods (Market Infrastructure section).

- **CurveBuilder and rate risk:** quote retention/repricing, sparse-grid policy, calibration diagnostics, quote-to-zero Jacobian, node/key-rate/quote sensitivities, and rate P&L explanation (Sections 5.2–5.3).
- **Credit foundations:** hazard/survival/default probabilities, CDS legs/par spreads, CS01-style sensitivity, jump-to-default diagnostics, and sequential bootstrap (Credit Foundations subsection).
- **Research validation:** CPCV, PBO, Reality Check, SPA, consolidated screens, leakage diagnostics, and multiverse/specification analysis (Research Validation section).
- **Research-data contracts:** deterministic snapshots, local store/cache/freshness/offline replay, and point-in-time observation/availability/revision views (Data, Point-in-Time Information and Provenance section).
- **Experiments and reporting:** configuration/data/code hashes, seed, environment/package identity, append-only registry, artifact lineage/stale propagation, and HTML/Markdown/JSON reports (Snapshots and Experiment Identity subsection).
- **Cross-domain utilities:** generic calibration/sensitivities, scenarios, four cost-model families, robust/cost-aware portfolio tools, covariance comparison, EVT/FHS/drawdown-at-risk, copulas, attribution, regimes, model comparison, Monte Carlo variance reduction, and capacity diagnostics (Cross-Domain Research Utilities and Decision Layer subsections).
- **Adapters:** registry/protocols for internal data, pricing, risk, execution, cost, and optimizer components (Cross-Domain Research Utilities and implications material).
- **Stability, CLI, and release gates:** public-API signature checks, native result serialization, the `info`, `doctor`, and non-mutating `validate` commands, the declared Python 3.10–3.13 compatibility matrix, and explicit gates for quote repricing, chronology, lineage, and API compatibility (Distribution/API Controls and Extended Release Evidence).
- **Compatibility/preservation:** the 1.0 paper contract, 1.1 discovery/rates capabilities, 1.2 canonical API, hypothesis workflow, and fail-closed live-broker boundary remain preserved (Introduction, Hypothesis Discovery, and limitations material).

E Audit-Finding Closure Matrix

ID	Class	Final 1.3.0 treatment
FIND-001	Import weight	Optional visualisation, HTTP and ML stacks are lazy; a fresh package import no longer introduces those stacks itself.
FIND-002	Packaging	Matplotlib/Plotly/Jinja2, scikit-learn, requests and pypdf are optional extras rather than core Requires-Dist entries.
FIND-003	Critical risk	Key-rate triangles use asymmetric neighbouring widths; regression tests verify partition behaviour on uneven pillars.
FIND-004	Estimation quality	Vasicek calibration exposes $se(\phi)$ and delta-method $se(\kappa)$; finite-sample mean-reversion uncertainty remains documented as an estimator property.
FIND-005	Information	<code>summary_metrics</code> exposes an Omega threshold instead of forcing the zero-threshold identity with Profit Factor.
FIND-006	GP contract	White-kernel noise is fixed at the requested value and requested/fitted noise are both reported.
FIND-011	Critical statistic	PSR/DSR operate on the per-observation Sharpe scale.
FIND-012	Compatibility	Removed statsmodels keyword arguments are not transmitted; the legacy ASR argument is retained only as a compatibility surface.
FIND-013	Convention	RSI-14 defaults to Wilder smoothing and exposes SMA explicitly.
FIND-014	Critical calibration	Intrinsic-boundary implied-volatility inversion raises a named non-identifiability error.
FIND-015	Factor stability	SVD/PCA signs are oriented deterministically using the largest absolute loading.

F Release-Candidate Finding Dispositions

Finding	Area	Final treatment
FIND-001	Import weight	Optional visualization, HTTP and ML stacks are lazy; a fresh package import no longer introduces those stacks itself.
FIND-002	Packaging	Visualization/reporting, ML, provider and PDF stacks are optional extras rather than core requirements.
FIND-003	Critical risk	Key-rate triangles use asymmetric neighbouring widths; regression tests verify partition behaviour on uneven pillars.
FIND-004	Estimation quality	Vasicek calibration exposes uncertainty diagnostics while finite-sample limitations remain documented.
FIND-005	Information	Summary metrics expose an explicit Omega threshold rather than forcing a zero-threshold identity with Profit Factor.
FIND-006	GP contract	Observation noise is fixed when supplied and requested/fitted noise are both reported.
FIND-007–010	Disposition	These numbered review slots are not represented by reproducible release-contract changes in the supplied evidence. They are not counted as closed software fixes; the source record does not support assigning more specific dispositions per ID.
FIND-011	Critical statistic	PSR/DSR operate on the per-observation Sharpe scale.
FIND-012	Compatibility	Removed statsmodels keyword arguments are not transmitted; legacy compatibility surfaces are retained where required.
FIND-013	Convention	RSI-14 defaults to Wilder smoothing and exposes SMA explicitly.
FIND-014	Critical calibration	Intrinsic-boundary implied-volatility inversion raises a named non-identifiability error.
FIND-015	Factor stability	SVD/PCA signs are oriented deterministically using the largest absolute loading.

Across FIND-001–FIND-015, eleven findings have documented final treatments in the supplied artifact and four identifiers (FIND-007–010) lack a reproducible closure record. The latter are disclosed explicitly and excluded from the closed-fix total rather than back-filled with unsupported dispositions.

G Packaging Profiles

The final distribution separates a small scientific core from capability extras. The core contains NumPy, pandas, SciPy, statsmodels, Pydantic and PyYAML. Research teams can then add visualisation/reporting, machine learning, HTTP providers, PDF literature ingestion, optimisation, volatility or data-format extras according to their controlled environment. This separation reduces mandatory transitive dependencies and makes air-gapped or regulated deployments easier to reason about.

```

pip install asrquant
pip install "asrquant[visualization]"
pip install "asrquant[ml]"
pip install "asrquant[providers]"
pip install "asrquant[literature]"

```

H Selected Numerical Contracts

H.1 Key-rate partition

For pillar t_i with neighbours t_{i-1} and t_{i+1} , the corrected triangular weight is piecewise asymmetric:

$$w_i(t) = \begin{cases} \frac{t - t_{i-1}}{t_i - t_{i-1}}, & t \in [t_{i-1}, t_i], \\ \frac{t_{i+1} - t}{t_{i+1} - t_i}, & t \in [t_i, t_{i+1}], \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

The left and right distances are not replaced by their maximum. This is the local condition needed for neighbouring tents to form a partition over an irregular grid.

H.2 PCA orientation

If v_j is a principal loading vector and $k = \arg \max_i |v_{ij}|$, the release applies

$$\tilde{v}_j = \text{sign}(v_{kj})v_j, \quad (9)$$

with a positive anchor. Scores receive the same sign, preserving reconstruction exactly while producing a stable reporting orientation.

H.3 Vasicek uncertainty

With the discretised AR(1) representation $r_{t+1} = a + \phi r_t + \varepsilon_{t+1}$ and $\kappa = -\log(\phi)/\Delta t$, the reported delta-method approximation is

$$\text{se}(\kappa) \approx \frac{\text{se}(\phi)}{\phi \Delta t}. \quad (10)$$

This diagnostic does not eliminate finite-sample bias; it makes the local identification uncertainty observable to the user.

I Data Dictionary and Provenance

Artifact	Role in this manuscript
Curve quote fixture	deterministic deposit/swap inputs for curve and quote-risk examples
Synthetic validation panel	fixed-seed strategy returns for PBO/Reality Check/SPA illustration
Credit spread fixture	deterministic CDS spreads and recovery for hazard bootstrap
Point-in-time timeline	conceptual availability/revision contract
Synthetic backtest	fixed-seed illustration of explicit implementation costs
Local validation record	336-test, import, API, example and distribution evidence

All quantitative figures used for release mechanics are deterministic or fixed-seed demonstrations. They are not presented as forecasts or live strategy results.

J Reproducibility Checklist

- Software target recorded as ASRQuant 1.3.0.
- Manuscript title, date, authors, release artifact, and correspondence stated.
- Point-in-time data distinguishes observation, availability, and revision timestamps.
- Hypothesis evidence and novelty status remain separate.
- Curve build exposes quote repricing and Jacobian diagnostics.
- Quote-space risk is distinguished from internal node risk.
- Selection-risk diagnostics include CPCV, PBO, Reality Check, SPA, leakage, and multiverse tools.
- Experiment fingerprints, data snapshots, and artifact lineage are represented explicitly.
- Backtest chronology and implementation costs remain explicit.
- Local validation counts trace to the 1.3 release validation record.
- Hosted release gates are not reported as locally passed.
- AI-use disclosure states final drafting and editorial polishing of the document only.